

Atividade 1 - Mineração de Dados

Gabriel M. Brum, Pedro H. M. Gatti

Programa da Pós-Graduação em Ciência da Computação (PPGCC) - UNESP

5 de outubro de 2026

1 Introdução

Nesta atividade, escolheu-se um conjunto de dados com ao menos 4 classes e aplicou-se o pré-processamento adequado. Em seguida, compararam-se os algoritmos de árvore de decisão, *bagging*, *boosting* e *random forest* por meio de uma técnica de validação e de uma métrica de desempenho, e ajustaram-se os hiperparâmetros do melhor modelo para verificar se o desempenho melhora. Por fim, o problema multiclasse foi transformado em problemas binários pelas estratégias OVA e OVO, e comparou-se qual delas se saiu melhor usando o algoritmo de árvore.

2 Dataset e pré-processamento

O conjunto *Car Evaluation*, disponibilizado no repositório UCI por Bohanec e Zupan em 1997 [12], tem origem em sistemas especialistas de decisão multicritério (ferramenta DEX) [1]. Na versão publicada, seis atributos categóricos descrevem diretamente a aceitabilidade de compra de um veículo em quatro classes (*unacc*, *acc*, *good* e *vgood*), e as 1728 instâncias cobrem todo o espaço discreto de combinações dos atributos. Não há valores nulos nem instâncias duplicadas. A distribuição de classes (Tabela 1) é fortemente desbalanceada.

Tabela 1: Distribuição das classes no conjunto de dados *Car Evaluation*.

Classe	Significado	Instâncias	% do total
<i>unacc</i>	Inaceitável	1210	70,023%
<i>acc</i>	Aceitável	384	22,222%
<i>good</i>	Bom	69	3,993%
<i>vgood</i>	Muito Bom	65	3,762%
Total	1728	100,000%	

Como todos os atributos, e a própria classe, têm ordem natural, aplicou-se *Ordinal Encoding* (Tabela 2).

3 Metodologia

Todos os experimentos usaram a biblioteca *scikit-learn* [9]. Devido ao desbalanceamento, a métrica decisora foi o *Macro F1-Score* ($F1_{\text{macro}}$), que atribui o mesmo peso a todas as classes [11], com validação estratificada em todas as etapas.

Tabela 2: Atributos, valores originais (em ordem crescente) e codificação ordinal aplicada.

Atributo	Valores originais	Codificação
buying, maint	<i>low, med, high, vhigh</i>	1, 2, 3, 4
doors	<i>2, 3, 4, 5more</i>	2, 3, 4, 5
persons	<i>2, 4, more</i>	2, 4, 5
lug_boot	<i>small, med, big</i>	1, 2, 3
safety	<i>low, med, high</i>	1, 2, 3
class (alvo)	<i>unacc, acc, good, vgood</i>	0, 1, 2, 3

3.1 Parte I: Comparação de modelos e otimização de hiperparâmetros

Os dados foram divididos em treino (70%) e teste (30%) de forma estratificada (`random_state=42`). A comparação entre modelos e o ajuste de hiperparâmetros usaram **somente o conjunto de treino**, e o conjunto de teste foi reservado para a avaliação final.

Com hiperparâmetros padrão, compararam-se a Árvore de Decisão [4], o *Bagging* [2], o AdaBoost [6] e a *Random Forest* [3] por validação cruzada estratificada de 5 *folds* (com *shuffle*), usando o $F1_{\text{macro}}$ médio como critério. O modelo vencedor foi ajustado com `GridSearchCV` (mesmas partições, `scoring='f1_macro'`), e os modelos padrão e ajustado foram comparados no conjunto de teste. Como o melhor escore de uma busca é otimista [5], o desempenho final é o medido no teste.

3.2 Parte II: Decomposição multiclasse em problemas binários

Usando `DecisionTreeClassifier(random_state=42)` como classificador base [7], o problema foi decomposto por **OVA** (`OneVsRestClassifier`: um classificador por classe contra as demais, 4 modelos) e **OVO** (`OneVsOneClassifier`: um classificador por par de classes, 6 modelos, com votação majoritária). Ambos foram avaliados por (i) *holdout* estratificado 70/30 (mesma partição da Parte I), com acurácia, $F1_{\text{macro}}$ e matrizes de confusão; e (ii) validação cruzada estratificada de 5 *folds* sobre o conjunto completo, com média e desvio padrão das métricas.

4 Resultados

4.1 Parte I: Comparação de modelos e otimização de hiperparâmetros

Tabela 3: Desempenho médio ($\pm$ desvio padrão entre *folds*) na validação cruzada de 5 *folds* sobre o conjunto de treino (1209 instâncias).

Modelo	Acurácia	$F1_{\text{macro}}$
Bagging	0,9710 $\pm$ 0,0128	0,9459 $\pm$ 0,0203
Árvore de Decisão	0,9719 $\pm$ 0,0109	0,9397 $\pm$ 0,0288
Random Forest	0,9661 $\pm$ 0,0076	0,9132 $\pm$ 0,0273
AdaBoost	0,8528 $\pm$ 0,0261	0,6702 $\pm$ 0,0584

A acurácia é semelhante entre Árvore, *Bagging* e *Random Forest*, pois é dominada pela classe *unacc*, e o $F1_{\text{macro}}$ é que os diferencia (Tabela 3). O *Bagging* foi selecionado, mas sua vantagem sobre a árvore isolada (0,0062) é menor que o desvio entre *folds*, de modo que este protocolo não permite distingui-los. O AdaBoost teve o pior resultado, provavelmente porque seu aprendiz fraco padrão (árvore de profundidade 1) depende de um único atributo e não captura as interações entre atributos que determinam a classe. A *Random Forest* ficou abaixo dos demais no $F1_{\text{macro}}$,

possivelmente porque cada nó considera apenas 2 dos 6 atributos, todos relevantes aqui; ambas são hipóteses não testadas, e valem para a configuração padrão.

Tabela 4: Grade de hiperparâmetros do *Bagging* (576 combinações) e melhor configuração. O prefixo `estimator__` refere-se à árvore base.

Hiperparâmetro	Valores testados	Melhor
<code>estimator__criterion</code>	gini, entropy	entropy
<code>estimator__max_depth</code>	None, 10	None
<code>estimator__min_samples_leaf</code>	1, 2	1
<code>estimator__class_weight</code>	None, balanced	balanced
<code>n_estimators</code>	100, 300	100
<code>max_samples</code>	0,6; 0,8; 1,0	0,8
<code>max_features</code>	4, 5, 6	6
<code>bootstrap</code>	True, False	True

A melhor configuração (Tabela 4) elevou o $F1_{\text{macro}}$ da validação interna de 0,9459 para 0,9653 ($\pm 0,0162$) nas mesmas partições. Contudo, as 10 melhores configurações diferem em apenas 0,005, menos que o desvio entre *folds* (0,013 a 0,021), o que indica baixa sensibilidade aos hiperparâmetros. A configuração escolhida é coerente com o método: árvores sem poda são aprendizes instáveis, cuja variância o *Bagging* reduz [2], e `class_weight='balanced'` compensa o desbalanceamento [8].

Tabela 5: Recall e F1 por classe no conjunto de teste (519 instâncias), antes e depois do ajuste. Acurácia global: 0,9692 $\rightarrow$ 0,9730.

Classe	Suporte	Recall		F1	
		Antes	Depois	Antes	Depois
<i>unacc</i>	363	0,9917	0,9945	0,9904	0,9904
<i>acc</i>	115	0,9478	0,9391	0,9437	0,9515
<i>good</i>	21	0,7143	0,8095	0,7895	0,8500
<i>vgood</i>	20	0,9500	0,9500	0,9048	0,9048
Macro	519	0,9010	0,9233	0,9071	0,9242

No conjunto de teste (Tabela 5), o ajuste melhorou o $F1_{\text{macro}}$ e a acurácia, com ganho concentrado na classe *good*: o recall passou de 15 para 17 acertos em 21 instâncias, e as confusões de *good* com *acc* caíram de 4 para 2 (Figura 1). Esse resultado é coerente com `class_weight='balanced'`, mas deve ser lido com cautela: corresponde a apenas 2 acertos a mais no total (505 contra 503), e uma única instância de *good* ou *vgood* altera o recall em quase 5 pontos percentuais. A diferença entre o $F1_{\text{macro}}$ de teste (0,9242) e o da validação interna (0,9653) é esperada pelo viés de seleção [5].

4.2 Parte II: Decomposição multiclasse com OVA e OVO

O OVO superou o OVA nos dois esquemas de validação e nas duas métricas (Tabela 6). A diferença é maior no $F1_{\text{macro}}$ do que na acurácia, o que indica que o ganho está nas classes minoritárias: no teste, o OVO acertou 19 das 21 instâncias de *good*, contra 13 do OVA, e 14 dos 21 erros do OVA foram predições *unacc* (Figura 2). Uma explicação plausível é que, no OVA, os classificadores de *good* e *vgood* enfrentam subproblemas muito desbalanceados (cerca de 4% contra 96%), o que favorece a classe majoritária [8], enquanto no OVO cada classificador é treinado apenas com duas classes [7]. Como o classificador base não teve hiperparâmetros

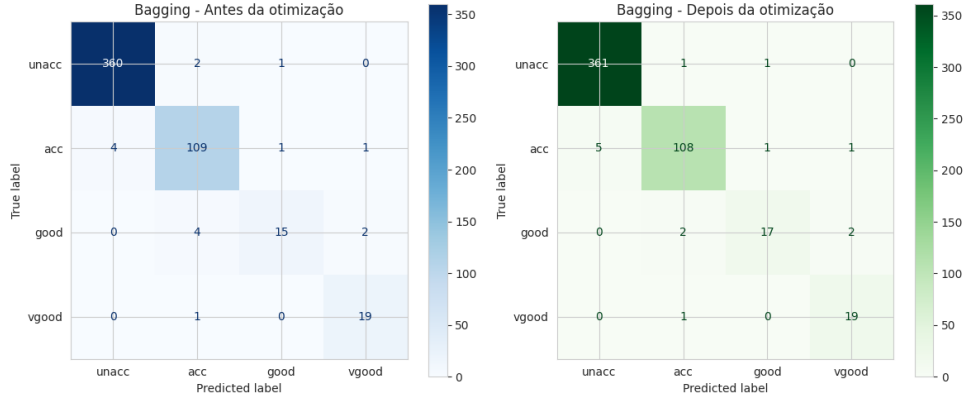


Figura 1: Matrizes de confusão do *Bagging* no conjunto de teste, antes (esquerda) e depois (direita) do ajuste. Linhas: classe real; colunas: classe predita.

Tabela 6: Desempenho de OVA e OVO com Árvore de Decisão. Na validação cruzada, média ($\pm$ desvio padrão) entre os 5 *folds*.

Validação	Métrica	OVA	OVO
<i>Holdout</i> 70/30	Acurácia	0,9595	0,9750
<i>Holdout</i> 70/30	$F1_{\text{macro}}$	0,9089	0,9575
CV 5 <i>folds</i>	Acurácia	$0,9687 \pm 0,0081$	$0,9821 \pm 0,0085$
CV 5 <i>folds</i>	$F1_{\text{macro}}$	$0,9272 \pm -$	$0,9592 \pm -$

ajustados, a conclusão vale para essa condição, já que o OVA pode se igualar ao OVO quando o classificador base é bem ajustado [10].

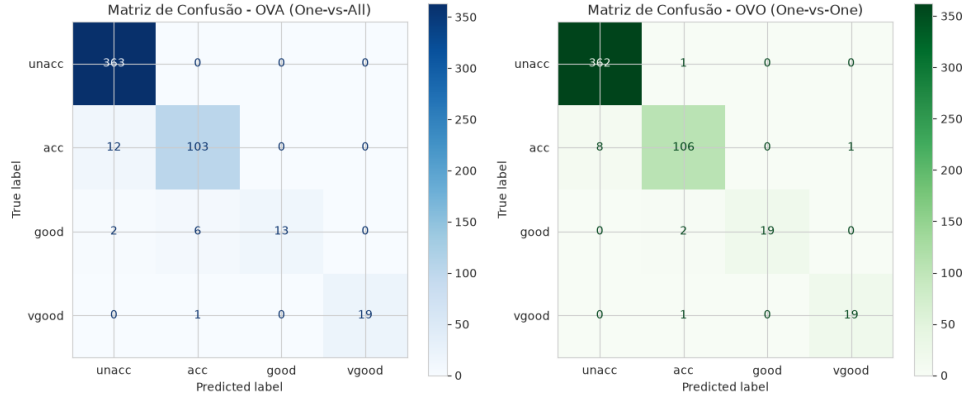


Figura 2: Matrizes de confusão de OVA (esquerda) e OVO (direita), com Árvore de Decisão como classificador base, no conjunto de teste. Linhas: classe real; colunas: classe predita.

5 Conclusão

Na Parte I, o *Bagging* obteve o maior $F1_{\text{macro}}$, equivalente ao da árvore isolada dentro da variabilidade entre *folds*, enquanto AdaBoost e *Random Forest*, em configuração padrão, ficaram abaixo. O ajuste de hiperparâmetros trouxe ganho pequeno no teste, concentrado na classe *good* e equivalente a apenas 2 acertos, com baixa sensibilidade aos hiperparâmetros. Na Parte II, o OVO foi superior ao OVA, sobretudo nas classes minoritárias. Na mesma partição de teste, o

OVO com árvore e o *Bagging* ajustado acertaram praticamente o mesmo número de instâncias (506 e 505).

Como limitações, as classes *good* e *vgood* têm cerca de 20 instâncias no teste, de modo que diferenças da ordem de centésimos devem ser interpretadas com cautela, e o conjunto de dados é completo e determinístico, o que impede extrapolar os resultados para dados ruidosos. Como trabalho futuro, sugere-se usar validação cruzada aninhada ou repetida com testes pareados e ajustar também os demais algoritmos para uma comparação em condições equivalentes.

Referências

- [1] Marko Bohanec and Vladislav Rajkovič. Knowledge acquisition and explanation for multi-attribute decision making. In *Proceedings of the 8th International Workshop on Expert Systems and their Applications*, pages 59–78, Avignon, France, 1988.
- [2] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.
- [3] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [4] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth & Brooks/Cole, Monterey, CA, 1984.
- [5] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.
- [6] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
- [7] Mikel Galar, Alberto Fernández, Edurne Barrenechea, Humberto Bustince, and Francisco Herrera. An overview of ensemble methods for binary classifiers in multi-class problems: Experimental study on one-vs-one and one-vs-all schemes. *Pattern Recognition*, 44(8):1761–1776, 2011.
- [8] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.
- [9] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [10] Ryan Rifkin and Aldebaro Klautau. In defense of one-vs-all classification. *Journal of Machine Learning Research*, 5:101–141, 2004.
- [11] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009.
- [12] Blaž Zupan, Marko Bohanec, Ivan Bratko, and Janez Demšar. Machine learning by function decomposition. In *Proceedings of the 14th International Conference on Machine Learning (ICML-97)*, pages 421–429, Nashville, TN, 1997.